

1. Pentesting

El **pentesting** (penetration testing o prueba de penetración) es una de las formas de corroborar el funcionamiento correcto de un sistema en cuanto a seguridad. Parte de la premisa de que *ya existen vulnerabilidades* y trata de probar que esas vulnerabilidades existen.

! Importante — remarcado en clase

Esta unidad corresponde al cierre del temario y **entra completa en el parcial**. El profe lo remarcó: “*Esto no es así un corolario, sino que va a entrar en el parcial.*”

— dicho en clase Lo más importante a saber de memoria es la **metodología de hipótesis de falla** (los pasos en orden) y la idea de que el pentesting **no garantiza seguridad**.

1.1. Formas de verificar un sistema: verificación formal vs. pentesting

Hay varias formas de corroborar el funcionamiento correcto de un sistema. Dos de ellas:

Definición: Verificación formal

Manera de comprobar **matemáticamente** que un sistema cumple con las restricciones que le imponemos. Esquema:

- **Precondiciones** → hipótesis sobre el estado del sistema (por ejemplo: el sistema está en un estado seguro).
- Se ejecutan las operaciones del sistema ($Op_1 \dots Op_n$) sobre un input dado.
- **Poscondiciones** → resultado de aplicar esas operaciones al input.
- **Requerimiento**: las poscondiciones cumplen las restricciones pedidas.

Sirve para probar matemáticamente que un sistema hace lo que dice hacer.

Características de la verificación formal según el profe:

- Es **súper compleja**: con un par de variables y operaciones ya es difícil de seguir; hay que mapear todas las posibilidades, precondiciones e interacciones entre componentes.
- Es **muy costosa y extensa**; a veces termina siendo **impráctica**: puede llevar más tiempo hacer la verificación formal que hacer el propio sistema.
- Peor aún cuando el sistema **evoluciona rápido**: antes de terminar la verificación ya quedó obsoleta y hay que hacer otra.
- Es realizable a nivel de funciones (como las viejas pruebas de escritorio: pruebo un input, otro input, qué pasa si le paso null), pero a nivel de un sistema completo se vuelve poco práctica.

Definición: Prueba de penetración (pentesting)

Prueba para verificar que un sistema cumple con ciertas restricciones.

- **Precondiciones** → hipótesis sobre el estado del sistema y la **existencia de una vulnerabilidad**.
- **Poscondiciones** → sistema comprometido.
- **Ejecución**: aplicar pruebas para intentar mover al sistema del estado inicial al estado

comprometido.

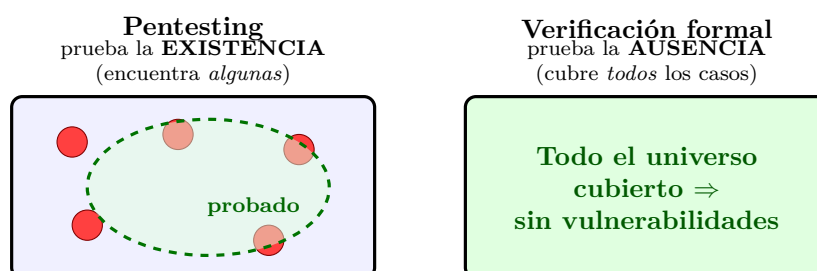
Es mucho más **económica, simple y eficiente** que la verificación formal, y es lo que hacen las empresas. Permite probar primero las áreas/componentes más críticos y después los menos críticos.

Similitudes y diferencias

- Ambas tratan de probar la **existencia** de vulnerabilidades.
- La **prueba de penetración NO prueba la ausencia** de vulnerabilidades.
- La **verificación formal SÍ prueba la ausencia**, pero para ello debe incluir **TODOS** los factores posibles (factores externos).
- En la práctica, la verificación formal prueba ausencia de vulnerabilidades en determinado algoritmo, programa o ambiente, **no en un sistema completo**. Se ignoran cosas como la **instalación** y el **uso**, el sistema operativo donde se instala, etc.

! Importante — remarcado en clase

El profe lo conecta con Dijkstra: el testing **puede mostrar la presencia** de bugs o vulnerabilidades, **pero no prueba su ausencia**. Esto va en total concordancia con el pentesting.



Cada punto rojo es una vulnerabilidad real. El pentesting solo cubre una porción del espacio (zona punteada): si no encuentra nada, **no** prueba que no haya fallas fuera de lo probado. La verificación formal cubre todos los casos y sí prueba la ausencia, pero requiere incluir **TODOS** los factores y por eso en la práctica se acota a un algoritmo/programa/ambiente, no a un sistema completo. Por esto el pentesting **no garantiza la seguridad**.

△ Cuidado — error típico

Que la poscondición sea “no encontré nada” / “no se dio la hipótesis” **no significa** que no haya alguna falla. Solo significa que el sistema **no está comprometido bajo las condiciones de esa prueba**. Puede haber otros vectores de ataque que no se probaron.

1.2. Objetivos y marco ético (ethical hacking)

El objetivo es **probar la eficacia de los controles de seguridad de un sistema**, intentando violar la **política de seguridad** existente. Requiere ejecutar técnicas similares a las de un atacante; por eso se lo llama **ethical hacking**.

- Generalmente hay un **equipo designado** que sabe de seguridad y conoce las implicancias. Hay que **simular ser un atacante** pero adoptando un comportamiento ético: no robar datos de clientes, no hacer compras a nombre de otra persona, etc.

- Es deseable que lo haga **alguien que no haya intervenido en la construcción** del sistema (análogo a QA: el desarrollador hace pruebas unitarias / de iteración / regresiones, y después un QA trata de romperlo con inputs inválidos, clics inesperados, etc.).
- Es **análogo a pruebas manuales** del sistema y **prueba al sistema como un todo**, no solo aspectos técnicos.
- **NO reemplaza un buen diseño e implementación**: estos tienen que estar de base y ser un requerimiento del sistema.

1.3. Metodología informal (preparación)

Vista como la etapa de preparación previa:

- **Determinar y cuantificar objetivos** (ej.: obtener información de clientes).
- Encontrar cierta cantidad de vulnerabilidades, o buscarlas durante un período de tiempo. El estudio se conduce **desde el punto de vista de un atacante**.
- Definir alcance, tiempo, propósito; armar el **plan de ataque**; elegir herramientas; seleccionar el equipo e identificar a quiénes van a participar y a estar al tanto (a veces **no se avisa**; lo del *lower line* / aviso a desarrolladores y equipos operativos se decide acá).
- Definir una condición de **objetivo** (éxito/no éxito). El ataque termina al lograr el objetivo o al vencerse el tiempo definido.
- **Estudiar y categorizar los hallazgos**: el éxito de una prueba de penetración proviene del **análisis de los hallazgos**. Es un proceso **cíclico**: los hallazgos se reincorporan al ciclo, permitiendo detectar problemas recurrentes y corregir sistemáticamente familias de problemas (una misma vulnerabilidad puede repetirse en varios componentes por usar la misma dependencia o forma de desarrollo).

1.4. Metodología de Hipótesis de Falla (LOS PASOS)

! Importante — remarcado en clase

“Esta metodología es importante, súper importante que la entiendan y la sepan; es la base de lo que hoy se usa para testear.”

— dicho en clase Es la base para equipos de Red Team, AppSec e InfraSec. Para sistemas medianos o grandes hoy se exige tener algún tipo de pentesting, sobre todo en los componentes *core*, donde se pide pasar el pentesting sin vulnerabilidades críticas o altas (definidas por los CVE; importancia de la organización **MITRE**).

Los **pasos en orden** (tal como aparecen en el PDF):

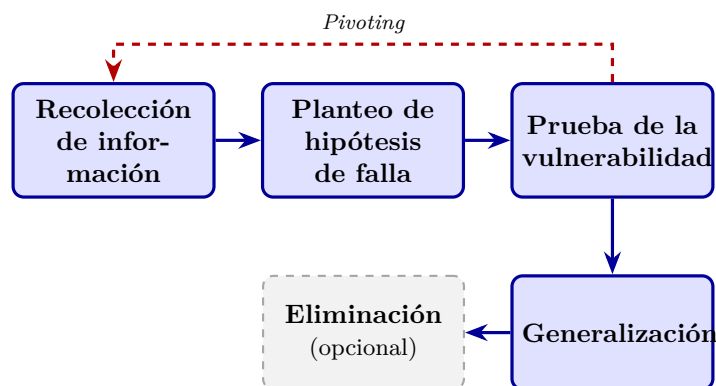
1. **Recolección de información** — para conocer el sistema y su funcionamiento.
2. **Hipótesis (Planteo de hipótesis de falla)** — asumir la existencia de vulnerabilidades concretas.
3. **Prueba** — probar las vulnerabilidades.
4. **Generalización** — generalizar patrones y hallar otras vulnerabilidades del mismo tipo.
5. **Eliminación (opcional)** — determinar los pasos necesarios para eliminarlas.

► En el parcial

! Foco de parcial (2025-2Q): los pasos del pentesting. El orden correcto incluye explícitamente el “**Planteo de hipótesis de falla**”. Secuencia tipo:

Recolección de info → Planteo de hipótesis de falla → Prueba → Generalización → Eliminación

El profe lo llama conocimiento “enciclopédico”: hay que saberlo de memoria. **Equivocar el orden o los pasos PENALIZA.** Notar que la **Eliminación es opcional** (solo se incluyen recomendaciones).



Metodología de hipótesis de falla: los pasos en orden (memorizar). La Eliminación es opcional (línea punteada). El Pivoting reinyecta nueva información a la fase de recolección, haciendo el proceso cíclico.

1.4.1. Paso 1: Recolección de información

- **Componer un modelo del sistema y sus partes.** Hay que conocer bien el sistema para saber “dónde poner el ojo”.
- **Buscar discrepancias:** cuando el sistema evoluciona y la documentación no refleja esa evolución (la doc dice que se comporta de una forma y ya no es así) → posible punto de entrada.
- **Revisar interfaces:** conexiones con sistemas externos, proveedores, otros sistemas, bases de datos.
- Buscar **vulnerabilidades conocidas genéricas** compatibles con la tecnología que usa el sistema.
- Apoyarse en la **documentación** (cuanta más, mejor); prestar especial atención a **secciones poco especificadas o ambiguas** (donde pudo tomarse una decisión cuestionable → otra puerta de entrada).
- Ver **manejo de privilegios y tipos de cuentas:** enumerar usuarios, buscar cuentas *super-admin* (las que más posibilidades dan, no están limitadas), cuentas **huérfanas** o con **exceso de permisos**.
- Conocer el **ambiente:** nombres de usuarios/servidores, en qué servidores corre, configuración y **estructura de red**.

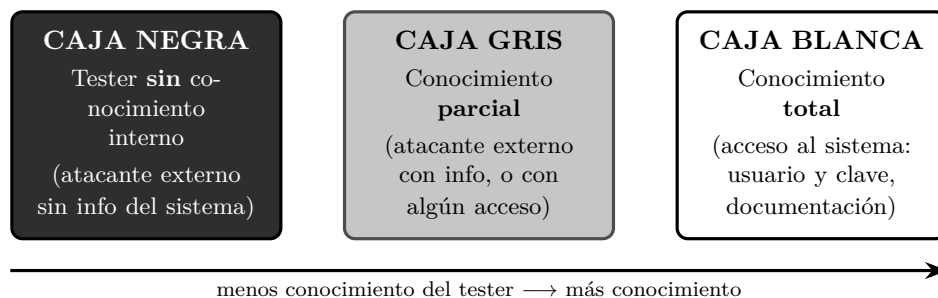
Punto de partida. Identifica la información inicial con la que cuenta el equipo (supuesto atacante). **Niveles incrementales:**

1. Atacante externo **sin** conocimiento del sistema.

2. Atacante externo **con** conocimiento del sistema.
3. Atacante **con acceso** al sistema (ya tiene usuario y contraseña).

Dependiendo del tipo de prueba, algunos niveles son irrelevantes: durante el diseño se ignora el primer nivel; en sistemas con registro abierto se ignora el segundo nivel.

Estos niveles de conocimiento inicial se corresponden con los tres tipos clásicos de prueba según cuánto sabe el tester del sistema:



Caja negra / gris / blanca según el conocimiento que tiene el tester del sistema. Se mapea con los niveles incrementales del punto de partida: externo sin info (negra), externo con info / acceso parcial (gris), acceso completo (blanca).

1.4.2. Paso 2: Hipótesis

Buscar posibles vulnerabilidades / combinaciones que expongan una vulnerabilidad:

- **Examinando políticas y procedimientos:** buscar inconsistencias, e inconsistencias entre políticas y mecanismos. Los procedimientos pueden **no cumplirse**.
- **Examinando implementaciones:** usar modelos de vulnerabilidades; revisar vulnerabilidades comunes/conocidas (*vulnerability scanning*, ej. **portmapper**); usar manuales (exceder límites, omitir pasos de las secuencias, etc.).
- **Examinando mecanismos:** pueden estar mal implementados, el ambiente donde corren puede introducir errores, o pueden no ser seguros.
- **Comparando con otros sistemas:** sistemas parecidos tienen problemas parecidos.

Áreas concretas que el profe menciona como candidatas de hipótesis:

- **Seguridad en APIs:** política de desarrollo de **microservicios** (más servicios = más “cajitas” atacables; si cada equipo no baja la misma política de seguridad, hay diseño menos robusto). **Software Supply Chain (API) Management:** hoy es **top 3** del OWASP Top Ten para apps web; abarca dependencias vulnerables, infiltración en flujos CI/CD con plugins/dependencias vulnerables, y ataques de confusión de paquetes (ej. casos de **npm**, donde se baja todo el árbol de dependencias transitivas).
- **RASP (Runtime Application Self Protection)** mal implementado: herramientas que controlan que los binarios del sistema no cambien durante la ejecución.
- **Verificar Federadores:** protocolos abiertos y antiguos (ej. OAuth para Google u otras integraciones) que las empresas implementan mal; vulnerabilidades conocidas sobre protocolos.
- **Software EOL (End Of Life):** fuera de soporte, sin actualizaciones de seguridad → “muy jugoso” a la hora de pentestear.

Resultado del paso 2: una lista de posibles vulnerabilidades que asumimos que tiene el sistema.

1.4.3. Paso 3: Prueba

- **Priorizar** la lista de posibles vulnerabilidades, según el objetivo de la prueba y, por lo general, según el **nivel de acceso requerido**.
- **Definir cómo probar** la existencia de la vulnerabilidad:
 - **Mejor manera:** analizar documentación o comportamiento.
 - **Otra manera:** intentar **explotarla** → último recurso, menos eficiente (hay que lograr el exploit como lo haría un atacante). **Excepción:** *Pivoting*.
- **Diseñar el test para que sea lo menos intrusivo posible:** algunas vulnerabilidades pueden **denegar el servicio**.
- **Proceso normal:** resguardar los datos de todo el sistema; documentar los requerimientos para detectar la vulnerabilidad; intentar detectarla.
- La prueba **DEBE ser REPETIBLE y CONCLUSIVA**: si funciona una sola vez y no se puede replicar, no concluye que haya una vulnerabilidad. Si yo lo puedo repetir, naturalmente lo puede repetir un atacante.

! Importante — remarcado en clase

Hoy por hoy se asume que **para demostrar una vulnerabilidad hay que tratar de explotarla** (las “PoC” del pentesting), siempre con cuidado, sobre todo en sistemas productivos: un exploit puede llevar el sistema a un estado inseguro, exponer datos, tirar abajo la aplicación (atentar contra Confidencialidad, Integridad o Disponibilidad — el “CID”). Ejemplos del profe: el caso **Chernóbil** (era una prueba controlada y terminó mal) y correr un **nmap** (escaneo/discovery de red) que tiró abajo la red de la oficina, afectando la disponibilidad.

△ Cuidado — error típico

En pentesting **NO siempre se intenta explotar** las vulnerabilidades encontradas; explotar es el *último recurso* y a veces alcanza con analizar documentación o comportamiento. (V/F, 2015.) En caja negra puede que sí se explote, pero el profe lo matiza: “intentar explotar es el último recurso, menos eficiente”.

Pivoting (excepción). Es probable que la existencia de una vulnerabilidad provea más información (ej. acceso al S.O.). En ese caso se **vuelve a la fase de recolección de información**. **Pivoting** es el uso de un **punto de acceso comprometido para continuar un ataque** (ej.: se compromete el firewall y desde allí se continúa atacando la red interna). Por eso se encadenan vulnerabilidades y se vuelve al ciclo.

1.4.4. Paso 4: Generalización

- A medida que las pruebas resultan exitosas, **emergen patrones**; se “conectan puntos”.
- A veces **dos vulnerabilidades combinadas** constituyen un problema grave. Ejemplo: cuenta de invitado habilitada permite conexión remota + buffer overflow local permite derechos de administrador ⇒ desde una cuenta de invitado se obtienen permisos de administrador.
- Es la **parte más importante** de la prueba: la “mano” del pentester es identificar cosas, combinarlas y producir el mayor impacto posible para dar *insight* al equipo/empresa que lo contrató.

1.4.5. Paso 5: Eliminación (opcional)

- Por lo general **solo se incluyen recomendaciones**, porque quien ejecuta la prueba no suele ser el diseñador/desarrollador (es el dueño del sistema quien decide qué hacer). A veces no se tiene la solución concreta (vulnerabilidades muy nuevas); a veces es tan simple como actualizar la versión de una dependencia, a veces es más abstracto (ej. “manejá bien la autorización”).
- Es importante que queden claros el **contexto, los detalles y el mecanismo de explotación** para: poder corregir el sistema; poder impedir/monitorear mientras tanto; verificar si fue explotado; y **reproducir la prueba** luego de corregir, para comprobar que ya no es vulnerable.
- Todo termina en un **reporte de pentesting** con las vulnerabilidades encontradas y las recomendaciones.

1.5. Herramientas y técnicas mencionadas

- **nmap**: programa de Linux que escanea interfaces y puertos, arma una topología de la red y hace un *discovery* (qué está abierto/cerrado, qué se conecta con qué). *Ojo*: mal usado puede tirar abajo la red.
- **Vulnerability scanning** (ej. portmapper) para vulnerabilidades conocidas.
- **Ingeniería social** (ver ejemplo de ataque externo).
- **CVE** (catálogo de vulnerabilidades conocidas, con su remediación) y la organización **MITRE**.
- **OWASP Top Ten** (referencia para apps web; Supply Chain como top 3).

1.6. Ejemplos

1.6.1. Ejemplo 1: Michigan Terminal System (MTS)

Sistema operativo que corre en **mainframes IBM 360/370**. Objetivo de la prueba: **obtener acceso a las estructuras de control** que no debería permitirse. Nivel inicial: **cuenta autorizada (nivel 3)**. Contaba con la **aprobación de los administradores**.

- **Paso 1 (Recolección)**: al correr un programa, la memoria se divide en segmentos. Segmentos **0–4**: supervisor, programas de sistema y estado, protegidos por hardware. Segmento **5**: área de trabajo del sistema e información del proceso (incluido el nivel de privilegio); el proceso **no** puede alterarlo. Segmentos **6+**: información del proceso, que el proceso modifica libremente. El segmento 5 está protegido por memoria virtual: en **modo sistema (kernel)** el proceso puede acceder/modificar y ejecutar funciones privilegiadas; en **modo usuario** el segmento 5 no se mapea. Un proceso corre en modo usuario y hace *system calls* (que corren en modo sistema). En un system call: se revisan los parámetros (especialmente que los punteros pertenezcan a zonas accesibles por el proceso); los parámetros se construyen como una **lista de direcciones** en el segmento de usuario; la lista se pasa como dirección en un registro.
- **Paso 2 (Hipótesis)**: ¿qué ocurriría si la dirección de un parámetro **apunta a la lista misma de parámetros**? Sin controles extra, un system call podría escribir la dirección del parámetro **después de haber pasado la validación**. Si se puede controlar qué valor escribir, técnicamente se podría leer/escribir cualquier zona del **segmento 5**.

- **Paso 3 (Prueba):** buscar un system call que use la convención estudiada, tenga al menos dos parámetros y altere uno. Se eligió `line input` (lee una línea de texto y retorna número de línea y longitud de línea). *Setup:* hacer que la dirección donde se almacena el número de línea sea la de la longitud de línea. *Ejecución:* el system call valida los parámetros (todos pertenecen al segmento del proceso); ejecuta `read input line`; la longitud de línea ingresada es el valor a escribir; el número de línea se guarda según la lista de parámetros (haciendo que sea una dirección del segmento 5, reescribiendo la dirección del otro parámetro), y así se escribe el valor en el segmento 5.
- **Paso 4 (Generalización):** no se puede escribir en los segmentos 0–4 (protegidos por hardware), pero en el segmento 5 el nivel de privilegio determina si pueden hacerse **llamadas de nivel supervisor** (en lugar de system calls), y una función de nivel supervisor **apaga la protección por hardware**. **Conclusión:** la vulnerabilidad permite a un atacante modificar cualquier dirección de cualquier segmento, controlando completamente la computadora.

1.6.2. Ejemplo 2: Ataque externo (ingeniería social)

Objetivo: determinar si las medidas de seguridad de una empresa eran efectivas para evitar que un atacante ingrese a un sistema. El test se enfoca en **políticas y procedimientos, tanto técnicos como no técnicos**, muy apalancado en **ingeniería social** (es más fácil vulnerar a las personas que a los sistemas).

- **Paso 1 (Recolección):** búsqueda en internet de nombres de empleados y directores y teléfono de la sucursal local; del **reporte anual público** (la empresa cotizaba en bolsa) reconstruyeron el organigrama, nombres de proyectos y personas. El tester se hizo pasar por nuevo empleado; aprendió que para enviar un documento se necesitaba **número de empleado y centro de costos**. Llamó a la secretaria del director: haciéndose pasar por empleado consiguió el número de empleado del director, y haciéndose pasar por auditor consiguió el centro de costos. Con eso solicitó enviar el listado a una consultora externa.
- **Paso 2 (Hipótesis):** los empleados nuevos no conocen todos los procesos y controles → es posible que un nuevo empleado brinde información sensible.
- **Paso 3 (Prueba):** el tester llamó a **RRHH** impersonando a la secretaria de un director (se quejó de que no le habían informado los nuevos ingresos y los pidió), obteniendo los nombres de los nuevos empleados. Luego llamó a esos nuevos empleados haciéndose pasar por **operador del centro de cómputos** y les brindó un “entrenamiento de seguridad” por teléfono, durante el cual obtuvo: tipos de sistemas usados, número de usuarios, **logins y claves** (incluso induciendo el cambio de una contraseña para conocerla). Se probó que los nuevos empleados no estaban al tanto de los procesos/controles.

△ Cuidado — error típico

Aunque el objetivo era otro, si se encuentra una falla distinta hay que **informarla igual**. Ejemplo del profe: el objetivo era probar la autenticación, pero se encontró que yendo a una URL del tipo `/info.txt` desde el browser de cualquier usuario quedaban expuestos datos de tarjetas → se debe reportar más allá del objetivo.

1.7. Limitaciones: el pentesting NO garantiza la seguridad

► En el parcial

! GANCHO/TRAMPA de parcial. El pentesting es una buena estrategia pero **NO garantiza la seguridad**. “*Garantizar la seguridad es algo que no se puede hacer jamás.*”

— dicho en clase

- Si en un ejercicio alguien afirma que algo “**garantiza la seguridad**”, es incorrecto y **hay penalización si se les escapa**.
- La alternativa con **más garantía** (aunque *tampoco* total) son las **pruebas / verificación formal**, hoy un área hipercéntrica por el código generado por **LLMs/agentes**.
- No confundir: la verificación formal *prueba la ausencia* de vulnerabilidades pero solo en un algoritmo/programa/ambiente acotado y debiendo incluir **TODOS** los factores; el pentesting *nunca* prueba ausencia.

Razones por las que el pentesting no garantiza seguridad (“Para discutir”):

- **No reemplaza** un buen diseño, especificación, implementación ni las pruebas (unitarias, de interacción, regresiones). Es una técnica importante para probar el sistema **luego de instalado**; idealmente no sería necesario, en la práctica sí.
- Encuentra **problemas introducidos por la interacción del sistema con los usuarios y el ambiente**, que suelen quedar fuera del análisis y pruebas normales.
- **No es sistémico / no es sistemático**: la calidad depende de la **capacidad de los testers para formular hipótesis** (metodología de hipótesis de falla). No provee una forma sistemática de revisar un sistema.
- **Cada test es un mundo aparte**: los resultados de un test sirven solo marginalmente para otros. Hay herramientas que automatizan *algunos* aspectos, pero no todos.
- Un pentesting **vale para ese sistema en ese momento**: si después cambia la API o el comportamiento (v2 → v3), la mayoría de las pruebas quedan obsoletas.

! Importante — remarcado en clase

Conclusión del profe: siempre hay que tener buen diseño, respetar los principios de seguridad y las especificaciones, y hacer buena cobertura de testing; el pentesting se suma *después* para comprobar que el diseño es bueno. “*Siempre algo aparece.*”

— dicho en clase

1.8. Lectura recomendada

- Matt Bishop, *Computer Security: Art and Science*, Capítulo 23 (1–2).
- **OSSTMM** — Open Source Security Testing Methodology Manual (<http://www.isecom.org/osstmm/>).